FIELD GUIDE

GitHub: from basic to senior

A practical ladder through GitHub and git. Every level builds on the one before it. Commands you actually run, habits that actually matter, and the mistakes that cost real time when you skip them.

LEVEL 1

Basics: account, repos, commits, push

LEVEL 2

Collaboration: branches, pull requests, reviews

LEVEL 3

Automation: Actions, releases, forks

LEVEL 4

Senior: security, governance, scale

Prepared for **Noah Otim** | 22 October 2026 | No em dashes were harmed in the making of this guide

CONTENTS

What is inside

Level 1. The Basics 03

Account, repositories, the four commands that run everything, README, issues

Level 2. Collaboration 05

Branches, pull requests, reviews, merge strategies, tags and releases

Level 3. Automation 07

GitHub Actions, CI pipelines, forks and upstream sync, GitHub CLI

Level 4. Senior Practice 09

Branch protection, secrets, Dependabot, triage, monorepos, org design

Cheat Sheet 11

The commands worth memorising, grouped by job

> How to use this guide: work top to bottom. Do not read Level 3 before Level 2 is muscle memory. Every section has one rule at the end that separates people who use GitHub from people who are governed by it.

LEVEL 1 · BEGINNER

The basics that everything else stands on

GitHub hosts git repositories. Git tracks every change to your files. That single idea, a permanent history you can rewind, is all you need to start. The rest is vocabulary and discipline.

Set up once

- Create an account at github.com, verify your email
- Install git locally, then tell it who you are

```
git config --global user.name "Noah Otim"
```

```
git config --global user.email "you@example.com"
```

- Use SSH keys instead of passwords for pushes
- The email on your commits must match your GitHub email or contributions will not count

The four commands that run git

```
git clone <url>
```

```
git add .
```

```
git commit -m "what changed and why"
```

```
git push
```

- Clone copies a repo to your machine
- Add stages changes, commit freezes them into history
- Push uploads your commits to GitHub
- Commit messages: imperative mood, under 70 characters, no "fixed stuff"

Reading the state

- `git status` tells you what changed and what is staged
- `git log --oneline -10` shows recent history
- `git diff` shows the actual line changes before you commit
- Green files are staged, red files are not. Committing is not the same as pushing

Repo hygiene from day one

- A README in the root: what it is, how to run it, what it depends on
- A `.gitignore` before your first commit, never after
- Never commit secrets: keys, tokens, .env files. Once pushed, assume leaked, rotate and purge
- Open an issue before starting work so the context survives you

Level 1 rule: commit small, commit often, and never commit anything you would not paste into a public chat.

LEVEL 1 · BEGINNER

Mistakes and their fixes

Every beginner hits these within the first month. Knowing the fix in advance saves the panic.

Undo things safely

- Committed but not pushed: `git reset --soft HEAD~1` keeps the changes staged
- Pushed a mistake to main: prefer a new commit that reverts, do not rewrite public history
- Wrong branch: `git stash`, switch branch, `git stash pop`
- Deleted a file by accident: `git checkout -- filename`

Conflicts, briefly

- A conflict means two people changed the same lines
- Open the file, find the `<<<<<<<` markers, keep the version you want, delete the markers
- Then `git add` the file and commit
- Do not resolve conflicts you do not understand. Ask the other author

GitHub vocabulary you need now

- **Repository** the project and its full history
- **Commit** one snapshot with a message
- **Branch** a parallel line of history
- **Remote** the GitHub copy your local repo talks to, usually called origin
- **Fork** someone else's repo copied into your account
- **Pull request** a request to merge your branch, with discussion attached

Level 1 habit: pull before you start work, `git pull`, so your branch is never built on stale code.

LEVEL 2 · INTERMEDIATE

Collaboration: branches and pull requests

One person can work straight on main. Two people cannot. Branches let work happen in parallel without breaking the shared line. Pull requests put a review and a conversation in front of every merge.

Branch discipline

```
git checkout -b feat/add-login
```

```
git push -u origin feat/add-login
```

- Name branches by intent: `feat/`, `fix/`, `docs/`, `chore/`
- One branch, one job. A branch that grows for a week becomes unmergeable
- Delete the branch after merge. Stale branches are lies about active work

The pull request loop

- Push your branch, open a PR on GitHub
- Title states the change, body states the why and how to test
- Link the issue it closes with `Closes #12`
- Request review, respond to every comment, merge when approved
- Keep PRs under roughly 400 changed lines. Big PRs get rubber-stamped, which is worse than no review

Merge strategies

- **Merge commit**: keeps all history, can clutter it
- **Squash and merge**: collapses the PR into one clean commit. Best default for feature branches
- **Rebase and merge**: linear history, but rewrites commits, so never use on shared branches

Reviews that are worth something

- Review the logic and the tests first, style last. Style belongs in an automated formatter
- Ask questions, do not dictate: "what happens if this input is empty?"
- Approve when you would run this code in production yourself
- Authors: respond to every comment, even if just "done" or "deliberate, here is why"

Level 2 rule: main should always work. Never merge anything you have not run, and never push directly to main once a repo has more than one contributor.

LEVEL 2 · INTERMEDIATE

Tags, releases and the project surface

History in commits is for you. Tags and releases are for everyone else. Issues and projects are how work stays visible when the team grows.

Tags and releases

```
git tag -a v1.2.0 -m "Add CSV import"
```

```
git push origin v1.2.0
```

- Semantic versioning: major for breaking changes, minor for features, patch for fixes
- Attach built artefacts to the GitHub Release so users do not clone to get a binary
- Write release notes as a person, not a changelog dump. What changed for the user

Issues, properly

- One problem per issue. "Fix stuff" cannot be closed honestly
- Template the body: what happened, what you expected, steps to reproduce
- Labels by type and priority, not vibes
- Close with commit messages: `Closes #14` links and auto-closes

GitHub Projects

- Boards (Todo, In progress, Done) for visible flow
- Table view with custom fields for dates and priority
- Automate: "when PR opened, move to In review"
- A weekly ritual beats a perfect board nobody updates

Useful intermediate moves

- `git cherry-pick` to lift one commit onto another branch
- `git rebase -i` to tidy your own commits before merge, never after
- Draft PRs to signal work in progress without inviting review yet
- Suggested changes in review: one-click commits for typo-level fixes

Level 2 habit: your PR description should let a colleague on holiday review it a week later with no Slack context.

LEVEL 3 · ADVANCED

Automation: GitHub Actions and CI

If a human runs the same check three times, it becomes a workflow file. GitHub Actions runs your tests, linters and deploys on every push, so the machine catches mistakes before reviewers do.

Anatomy of a workflow

- YAML files live in `.github/workflows/`
- **Trigger**: on push, on pull request, on schedule, on manual dispatch
- **Jobs**: sets of steps that run on an runner (ubuntu, windows, macos)
- **Steps**: shell commands or shared actions from the marketplace

What belongs in CI

- Tests and type checks, always
- Linters and formatters, so style debates die
- Build, to prove it compiles, not just passes tests
- Deploy to production only from main, gated on all of the above

A minimal test workflow

```
name: tests
on: [push, pull_request]
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- run: npm ci
- run: npm test
```

Secrets in Actions

- Store them under Settings, Secrets and variables, Actions
- Reference as `${{ secrets.API_KEY }}`, never in the YAML itself
- Fork PRs do not receive your secrets, by design. That is a feature
- Rotate any secret that ever appears in a log

Level 3 rule: if the pipeline is flaky, fix the pipeline before you trust it. A red build everyone ignores is worse than no build.

LEVEL 3 · ADVANCED

Forks, upstream and the wider workflow

Real work happens across repositories. You contribute to other people's projects, keep your fork current, and script your day with the GitHub CLI instead of clicking through the website.

Fork and contribute flow

```
git clone https://github.com/you/project
```

```
git remote add upstream https://github.com/owner/project
```

```
git fetch upstream
```

```
git merge upstream/main
```

- Fork on GitHub, clone your fork locally, add upstream as a second remote
- Branch from up-to-date upstream, not from a stale fork
- Sync before you branch, every single time
- Keep your fork's default branch behind the upstream, do not diverge it

GitHub CLI (gh)

```
gh repo create my-app --public
```

```
gh issue create --title "Login fails on Safari"
```

```
gh pr create --fill
```

```
gh pr merge --squash --delete-branch
```

```
gh run watch
```

- Everything clickable on GitHub is scriptable with gh
- Use it in the terminal so your workflow stays in one place
- `gh api` hits any REST endpoint when the CLI has no command

Submodules, briefly

- A repo pinned inside another repo
- Clone with `--recurse-submodules` or you get empty folders and confusion
- Update with `git submodule update --remote`, then commit the pointer
- Prefer package managers or monorepos when you can. Submodules are a last resort

Level 3 habit: when you do anything on GitHub manually twice, check whether gh or an Action can do it for you forever.

LEVEL 4 · SENIOR

Governance: protecting the main line

At senior level your job shifts from writing code to protecting the system that writes code. Rules on the repository, not heroics in the team chat, keep production safe.

Branch protection, minimum set

- Block force pushes on main and block its deletion
- Require a pull request with at least one review before merge
- Require status checks to pass, so CI is a gate, not decoration
- Include administrators in the rules, or the rules protect nobody
- Note: branch protection on private repos needs a paid GitHub plan. Public repos have it free

Rulesets, the newer tool

- Same job as branch protection, but stackable and targetable by pattern
- Block force push, block deletion, require PR, require signed commits
- Can restrict who can push, by team or role
- Use rulesets when a repo has many long-lived branches, like release lines

Codifying review quality

- `CODEOWNERS` file: automatic reviewers by path. Infra changes need the infra owner
- Require review from code owners on critical paths
- Require conversation resolution before merge
- Branch naming patterns in rulesets stop "asdf" branches reaching main

Incident response with git

- Revert is safer than rewrite on a shared branch: `git revert <sha>`
- Hotfix branches cut from the release tag, not from main, when main is already ahead
- Every incident ends with a fix that lands in CI so it cannot recur
- Blameless postmortems. The system failed, not the person

Level 4 rule: never rely on people remembering a rule. If it matters, encode it in branch protection, a ruleset or CI, then the rule survives holidays and turnover.

LEVEL 4 · SENIOR

Security and the codebase at scale

Senior engineers assume the supply chain is a target and the repo will outlive the current team. Both assumptions are correct often enough to design for.

Security posture

- **Secret scanning**: flags pushed keys. Enable push protection so GitHub blocks the push outright
- **Dependabot alerts and updates**: free for public repos, automatic PRs when dependencies have known vulnerabilities
- **Code scanning**: CodeQL finds injection and unsafe patterns in your own code
- Pin Actions to full commit SHAs, not tags, in anything security-relevant. Tags can be moved by a compromised publisher
- Review who has write access quarterly. Access never shrinks on its own

Scale patterns

- **Monorepo**: one repo, many packages. Great for atomic changes, needs path-based CI and CODEOWNERS to stay sane
- **Polyrepo**: many repos. Clean boundaries, but cross-repo changes need coordinated releases
- **Stacked PRs**: series of dependent branches for large refactors, reviewed in slices
- Archive repos instead of deleting. History has value even when code is dead
- Templates for issues and PRs raise the floor of every future contribution

Supply chain hygiene

- Lockfiles committed, always. No lockfile means every install can differ
- Prefer well-known Actions over obscure marketplace ones with 12 stars
- Read every new GitHub App you install. It sees your code

The senior habits

- Delete branches after merge, close stale issues weekly
- Write the ADR (architecture decision record) when you choose something non-obvious
- Review others' PRs first thing. It unblocks them, and your own queue clears behind it
- Mentor in PR comments. The comment outlives the conversation

Level 4 habit: every process you add should make the correct path the easy path. If the safe way is longer than the unsafe way, people will take the unsafe way quietly.

REFERENCE

Cheat sheet

The commands worth memorising. The rest you look up, which is also fine.

JOB	COMMAND
Start a repo	`git init` · `git clone <url>`
Stage and save	`git add .` · `git commit -m "message"`
Share	`git push -u origin branch-name`
Update local	`git pull`
New branch	`git checkout -b feat/thing`
Tidy own commits	`git rebase -i HEAD~3`
Undo last commit, keep changes	`git reset --soft HEAD~1`
Undo a public commit	`git revert <sha>`
Stash work in progress	`git stash` · `git stash pop`
Sync a fork	`git fetch upstream` · `git merge upstream/main`
Tag a release	`git tag -a v1.0.0 -m "notes"` · `git push origin v1.0.0`
PR from terminal	`gh pr create --fill`
Merge from terminal	`gh pr merge --squash --delete-branch`
Watch CI run	`gh run watch`
Find anything via API	`gh api /repos/{owner}/{repo}/actions/runs`

The five rules that cover 90 percent of situations

- Never force push a shared branch
- Never commit secrets, not even temporarily
- Branch per change, merge by PR, delete after
- Small PRs get real reviews, large ones get rubber stamps
- If you do it twice, automate it the third time